

Map和Set

【本节目标】

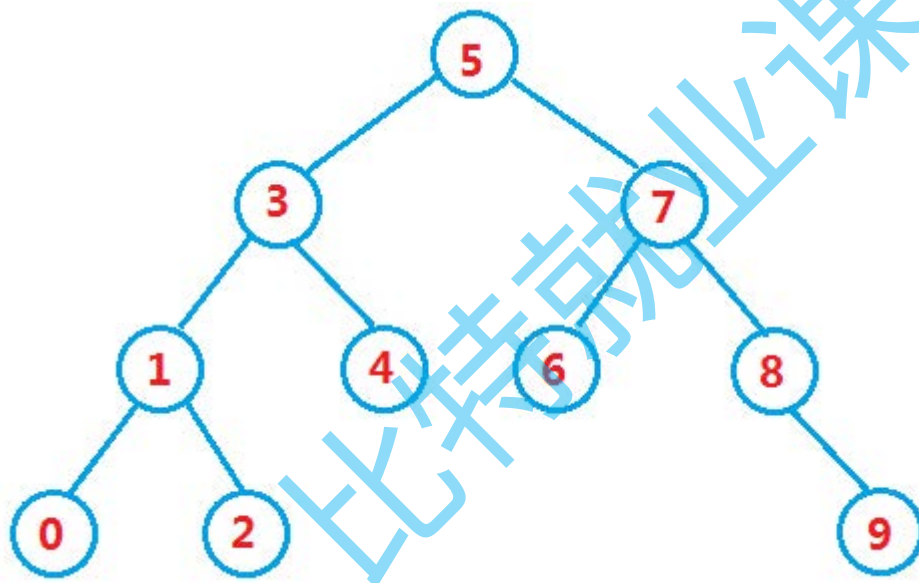
1. 掌握 Map/Set 及实际实现类 HashMap/TreeMap/HashSet/TreeSet 的使用
2. 掌握 HashMap 和 HashSet 背后的数据结构哈希表的原理和简单实现

1.搜索树

1.1 概念

二叉搜索树又称二叉排序树，它或者是一棵空树，或者是具有以下性质的二叉树：

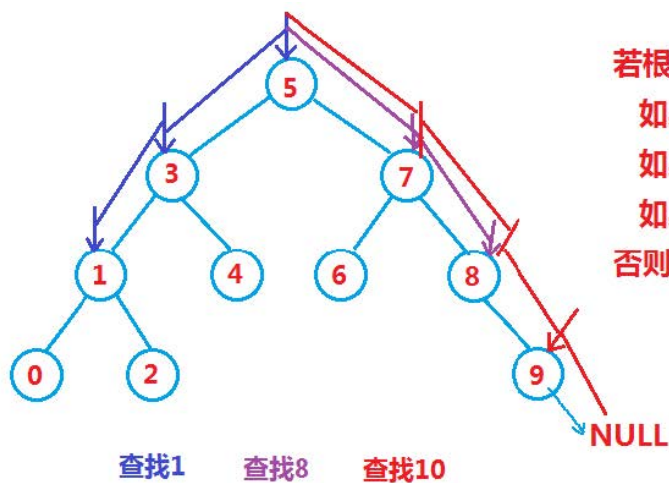
- 若它的左子树不为空，则左子树上所有节点的值都小于根节点的值
- 若它的右子树不为空，则右子树上所有节点的值都大于根节点的值
- 它的左右子树也分别为二叉搜索树



int[] array =

{5,3,4,1,7,8,2,6,0,9};

1.2 操作-查找



若根节点不为空：

如果根节点key == 查找key 返回true

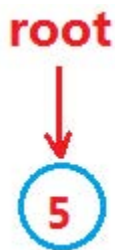
如果根节点key > 查找key 在其左子树查找

如果根节点key < 查找key 在其右子树查找

否则 返回false

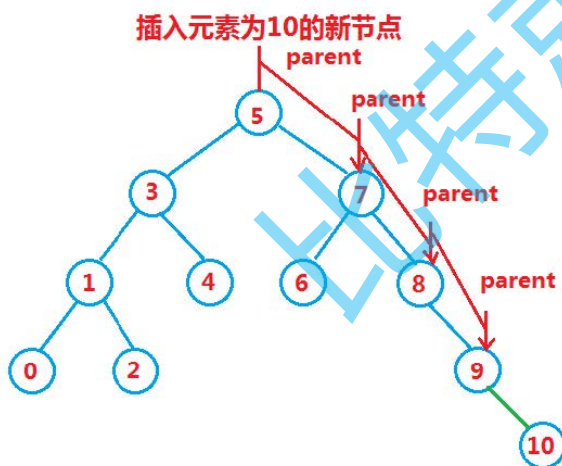
1.3 操作-插入

1. 如果树为空树，即根 == null，直接插入



如果是空树，直接插入，然后返回true

2. 如果树不是空树，按照查找逻辑确定插入位置，插入新结点



1. 按照二叉搜索树的性质，查找到插入结点的位置

root-->5 5<10 root = root->right parent = root

root-->7 7<10 root = root->right parent = root

root-->8 8<10 root = root->right parent = root

root-->9 9<10 root = root->right parent = root

2. 插入新结点

1.4 操作-删除 (难点)

设待删除结点为 cur，待删除结点的双亲结点为 parent

1. cur.left == null

1. cur 是 root，则 root = cur.right

2. cur 不是 root，cur 是 parent.left，则 parent.left = cur.right

3. cur 不是 root，cur 是 parent.right，则 parent.right = cur.right

2. cur.right == null

1. cur 是 root，则 root = cur.left

2. cur 不是 root，cur 是 parent.left，则 parent.left = cur.left

3. cur 不是 root, cur 是 parent.right, 则 parent.right = cur.left

3. cur.left != null && cur.right != null

1. 需要使用**替换法**进行删除, 即在它的右子树中寻找中序下的第一个结点(关键码最小), 用它的值填补到被删除节点中, 再来处理该结点的删除问题

1.5 实现

```
public class BinarySearchTree {
    public static class Node {
        int key;
        Node left;
        Node right;

        public Node(int key) {
            this.key = key;
        }
    }

    private Node root = null;

    /**
     * 在搜索树中查找 key, 如果找到, 返回 key 所在的结点, 否则返回 null
     * @param key
     * @return
     */
    public Node search(int key) {
        Node cur = root;
        while (cur != null) {
            if (key == cur.key) {
                return cur;
            } else if (key < cur.key) {
                cur = cur.left;
            } else {
                cur = cur.right;
            }
        }

        return null;
    }

    /**
     * 插入
     * @param key
     * @return true 表示插入成功, false 表示插入失败
     */
    public boolean insert(int key) {
        if (root == null) {
            root = new Node(key);
            return true;
        }

        Node cur = root;
```

```

Node parent = null;
while (cur != null) {
    if (key == cur.key) {
        return false;
    } else if (key < cur.key) {
        parent = cur;
        cur = cur.left;
    } else {
        parent = cur;
        cur = cur.right;
    }
}

Node node = new Node(key);
if (key < parent.key) {
    parent.left = node;
} else {
    parent.right = node;
}
return true;
}

/**
 * 删除成功返回 true, 失败返回 false
 * @param key
 * @return
 */
public boolean remove(int key) {
    Node cur = root;
    Node parent = null;
    while (cur != null) {
        if (key == cur.key) {
            break;
        } else if (key < cur.key) {
            parent = cur;
            cur = cur.left;
        } else {
            parent = cur;
            cur = cur.right;
        }
    }

    // 该元素不在二叉搜索树中
    if (null == cur) {
        return false;
    }

    /*
    根据cur的孩子是否存在分四种情况
    1. cur左右孩子均不存在
    2. cur只有左孩子
    3. cur只有右孩子
    4. cur左右孩子均存在

```

看起来有四种情况，实际情况1可以与情况2或者3进行合并，只需要处理是那种情况即可
除了情况4之外，其他情况可以直接删除
情况4不能直接删除，需要在其子树中找一个替代节点进行删除
*/
// 请同学们根据上课掌握内容，完成删除的关键部分代码
return true;

}

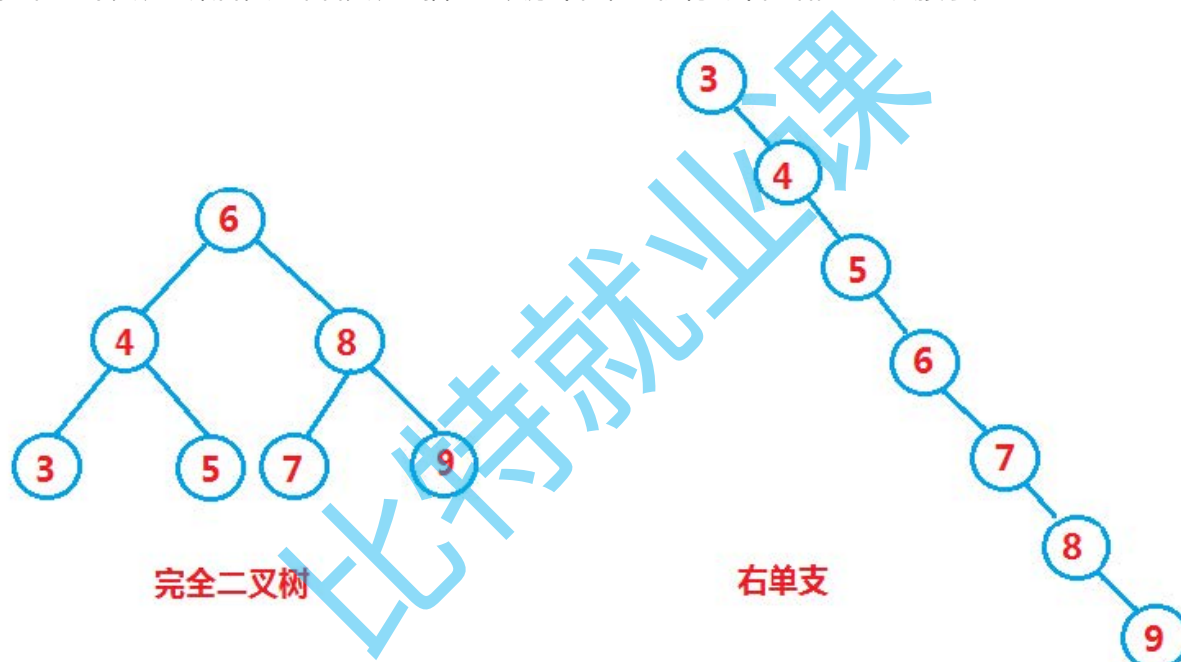
}

1.6 性能分析

插入和删除操作都必须先查找，查找效率代表了二叉搜索树中各个操作的性能。

对有n个结点的二叉搜索树，若每个元素查找的概率相等，则二叉搜索树平均查找长度是结点在二叉搜索树的深度的函数，即结点越深，则比较次数越多。

但对于同一个关键码集合，如果各关键码插入的次序不同，可能得到不同结构的二叉搜索树：



最优情况下，二叉搜索树为完全二叉树，其平均比较次数为： $\log_2 N$

最差情况下，二叉搜索树退化为单支树，其平均比较次数为： $\frac{N}{2}$

问题：如果退化成单支树，二叉搜索树的性能就失去了。那能否进行改进，不论按照什么次序插入关键码，都可以是二叉搜索树的性能最佳？

1.7 和 java 类集的关系

TreeMap 和 TreeSet 即 java 中利用搜索树实现的 Map 和 Set；实际上用的是红黑树，而红黑树是一棵近似平衡的二叉搜索树，即在二叉搜索树的基础之上 + 颜色以及红黑树性质验证，关于红黑树的内容后序再进行讲解。

2. 搜索

2.1 概念及场景

Map和set是一种专门用来进行搜索的容器或者数据结构，其搜索的效率与其具体的实例化子类有关。以前常见的搜索方式有：

1. 直接遍历，时间复杂度为 $O(N)$ ，元素如果比较多效率会非常慢
2. 二分查找，时间复杂度为 $O(\log_2 N)$ ，但搜索前必须要求序列是有序的

上述排序比较适合静态类型的查找，即一般不会对区间进行插入和删除操作了，而现实中的查找比如：

1. 根据姓名查询考试成绩
2. 通讯录，即根据姓名查询联系方式
3. 不重复集合，即需要先搜索关键字是否已经在集合中

可能在查找时进行一些插入和删除的操作，即动态查找，那上述两种方式就不太适合了，本节介绍的Map和Set是一种适合动态查找的集合容器。

2.2 模型

一般把搜索的数据称为关键字（Key），和关键字对应的称为值（Value），将其称之为Key-value的键值对，所以模型会有两种：

1. **纯 key 模型**，比如：

- 有一个英文词典，快速查找一个单词是否在词典中
- 快速查找某个名字在不在通讯录中

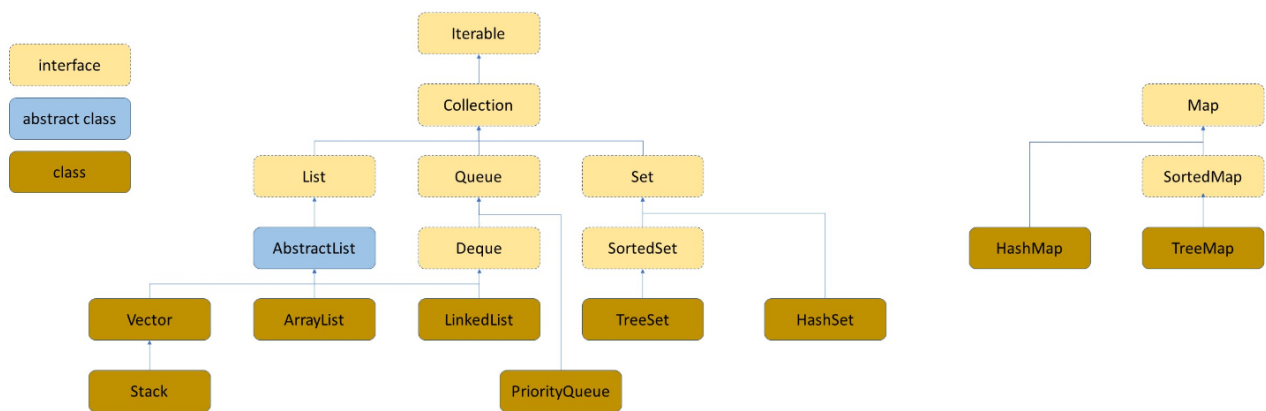
2. **Key-Value 模型**，比如：

- 统计文件中每个单词出现的次数，统计结果是每个单词都有与其对应的次数：<单词，单词出现的次数>
- 梁山好汉的江湖绰号：每个好汉都有自己的江湖绰号

而**Map中存储的就是key-value的键值对，Set中只存储了Key。**

3. Map 的使用

[Map 的官方文档](#)



3.1 关于Map的说明

Map是一个接口类，该类没有继承自Collection，该类中存储的是<K,V>结构的键值对，并且K一定是唯一的，不能重复。

3.2 关于Map.Entry<K, V>的说明

Map.Entry<K, V> 是Map内部实现的用来存放<key, value>键值对映射关系的内部类，该内部类中主要提供了<key, value>的获取，value的设置以及Key的比较方式。

方法	解释
K getKey()	返回 entry 中的 key
V getValue()	返回 entry 中的 value
V setValue(V value)	将键值对中的value替换为指定value

注意：Map.Entry<K,V>并没有提供设置Key的方法

3.3 Map 的常用方法说明

方法	解释
V get (Object key)	返回 key 对应的 value
V getOrDefault (Object key, V defaultValue)	返回 key 对应的 value, key 不存在, 返回默认值
V put (K key, V value)	设置 key 对应的 value
V remove (Object key)	删除 key 对应的映射关系
Set<K> keySet ()	返回所有 key 的不重复集合
Collection<V> values ()	返回所有 value 的可重复集合
Set<Map.Entry<K, V>> entrySet ()	返回所有的 key-value 映射关系
boolean containsKey (Object key)	判断是否包含 key
boolean containsValue (Object value)	判断是否包含 value

注意:

1. Map是一个接口, 不能直接实例化对象, 如果要实例化对象只能实例化其实现类TreeMap或者HashMap
2. Map中存放键值对的Key是唯一的, value是可以重复的
3. 在TreeMap中插入键值对时, key不能为空, 否则就会抛NullPointerException异常, value可以为空。但是HashMap的key和value都可以为空。
4. Map中的Key可以全部分离出来, 存储到Set中来进行访问(因为Key不能重复)。
5. Map中的value可以全部分离出来, 存储在Collection的任何一个子集合中(value可能有重复)。
6. Map中键值对的Key不能直接修改, value可以修改, 如果要修改key, 只能先将该key删除掉, 然后再来进行重新插入。
7. TreeMap和HashMap的区别【HashMap在课件最后会讲到】

Map底层结构	TreeMap	HashMap
底层结构	红黑树	哈希桶
插入/删除/查找时间复杂度	$O(\log_2 N)$	$O(1)$
是否有序	关于Key有序	无序
线程安全	不安全	不安全
插入/删除/查找区别	需要进行元素比较	通过哈希函数计算哈希地址
比较与覆写	key必须能够比较, 否则会抛出ClassCastException异常	自定义类型需要覆写equals和hashCode方法
应用场景	需要Key有序场景下	Key是否有序不关心, 需要更高的时间性能

3.4 TreeMap的使用案例

```
import java.util.TreeMap;
import java.util.Map;

public static void TestMap(){
    Map<String, String> m = new TreeMap<>();

    // put(key, value):插入key-value的键值对
    // 如果key不存在, 会将key-value的键值对插入到map中,返回null
    m.put("林冲", "豹子头");
    m.put("鲁智深", "花和尚");
    m.put("武松", "行者");
    m.put("宋江", "及时雨");
    String str = m.put("李逵", "黑旋风");
    System.out.println(m.size());
    System.out.println(m);

    // put(key,value): 注意key不能为空, 但是value可以为空
    // key如果为空, 会抛出空指针异常
    //m.put(null, "花名");
    str = m.put("无名", null);
    System.out.println(m.size());

    // put(key, value):
    // 如果key存在, 会使用value替换原来key所对应的value, 返回旧value
    str = m.put("李逵", "铁牛");

    // get(key): 返回key所对应的value
    // 如果key存在, 返回key所对应的value
    // 如果key不存在, 返回null
    System.out.println(m.get("鲁智深"));
    System.out.println(m.get("史进"));

    //GetOrDefault(): 如果key存在, 返回与key所对应的value, 如果key不存在, 返回一个默认值
    System.out.println(m.getOrDefault("李逵", "铁牛"));
    System.out.println(m.getOrDefault("史进", "九纹龙"));
    System.out.println(m.size());

    //containsKey(key): 检测key是否包含在Map中, 时间复杂度: O(logN)
    // 按照红黑树的性质来进行查找
    // 找到返回true, 否则返回false
    System.out.println(m.containsKey("林冲"));
    System.out.println(m.containsKey("史进"));

    // containValue(value): 检测value是否包含在Map中, 时间复杂度: O(N)
    // 找到返回true, 否则返回false
    System.out.println(m.containsValue("豹子头"));
    System.out.println(m.containsValue("九纹龙"));

    // 打印所有的key
    // keySet是将map中的key防止在Set中返回的
```

```

for(String s : m.keySet()){
    System.out.print(s + " ");
}
System.out.println();

// 打印所有的value
// values()是将map中的value放在collect的一个集合中返回的
for(String s : m.values()){
    System.out.print(s + " ");
}
System.out.println();

// 打印所有的键值对
// entrySet(): 将Map中的键值对放在Set中返回了
for(Map.Entry<String, String> entry : m.entrySet()){
    System.out.println(entry.getKey() + "---->" + entry.getValue());
}
System.out.println();
}

```

同学们可使用TreeMap来实例化Map，看看TreeMap和HashMap的不同。

4. Set 的说明

[Set 的官方文档](#)

Set与Map主要的不同有两点：Set是继承自Collection的接口类，Set中只存储了Key。

4.1 常见方法说明

方法	解释
boolean add (E e)	添加元素，但重复元素不会被添加成功
void clear ()	清空集合
boolean contains (Object o)	判断 o 是否在集合中
Iterator<E> iterator ()	返回迭代器
boolean remove (Object o)	删除集合中的 o
int size()	返回set中元素的个数
boolean isEmpty()	检测set是否为空，空返回true，否则返回false
Object[] toArray()	将set中的元素转换为数组返回
boolean containsAll(Collection<?> c)	集合c中的元素是否在set中全部存在，是返回true，否则返回false
boolean addAll(Collection<? extends E> c)	将集合c中的元素添加到set中，可以达到去重的效果

注意：

- 1. Set是继承自Collection的一个接口类
- 2. Set中只存储了key，并且要求key一定要唯一
- 3. TreeSet的底层是使用Map来实现的，其使用key与Object的一个默认对象作为键值对插入到Map中的
- 4. Set最大的功能就是对集合中的元素进行去重
- 5. 实现Set接口的常用类有TreeSet和HashSet，还有一个LinkedHashSet，LinkedHashSet是在HashSet的基础上维护了一个双向链表来记录元素的插入次序。
- 6. Set中的Key不能修改，如果要修改，先将原来的删除掉，然后再重新插入
- 7. TreeSet中不能插入null的key，HashSet可以。
- 8. TreeSet和HashSet的区别【HashSet在课件最后会讲到】

Set底层结构	TreeSet	HashSet
底层结构	红黑树	哈希桶
插入/删除/查找时间复杂度	$O(\log_2 N)$	$O(1)$
是否有序	关于Key有序	不一定有序
线程安全	不安全	不安全
插入/删除/查找区别	按照红黑树的特性来进行插入和删除	1. 先计算key哈希地址 2. 然后进行插入和删除
比较与覆写	key必须能够比较，否则会抛出ClassCastException异常	自定义类型需要覆写equals和hashCode方法
应用场景	需要Key有序场景下	Key是否有序不关心，需要更高的时间性能

4.2 TreeSet的使用案例

```
import java.util.TreeSet;
import java.util.Iterator;
import java.util.Set;

public static void TestSet(){
    Set<String> s = new TreeSet<>();

    // add(key): 如果key不存在，则插入，返回ture
    // 如果key存在，返回false
    boolean isIn = s.add("apple");
    s.add("orange");
    s.add("peach");
    s.add("banana");
    System.out.println(s.size());
    System.out.println(s);
}
```

```

isIn = s.add("apple");

// add(key): key如果是空, 抛出空指针异常
//s.add(null);

// contains(key): 如果key存在, 返回true, 否则返回false
System.out.println(s.contains("apple"));
System.out.println(s.contains("watermelen"));

// remove(key): key存在, 删除成功返回true
//      key不存在, 删除失败返回false
//      key为空, 抛出空指针异常
s.remove("apple");
System.out.println(s);

s.remove("watermelen");
System.out.println(s);

Iterator<String> it = s.iterator();
while(it.hasNext()){
    System.out.print(it.next() + " ");
}
System.out.println();
}

```

5. 哈希表

5.1 概念

顺序结构以及平衡树中, 元素关键码与其存储位置之间没有对应的关系, 因此在查找一个元素时, 必须要经过关键码的多次比较。顺序查找时间复杂度为 $O(N)$, 平衡树中为树的高度, 即 $O(\log_2 N)$, 搜索的效率取决于搜索过程中元素的比较次数。

理想的搜索方法: 可以不经过任何比较, 一次直接从表中得到要搜索的元素。如果构造一种存储结构, 通过某种函数(hashFunc)使元素的存储位置与它的关键码之间能够建立一一映射的关系, 那么在查找时通过该函数可以很快找到该元素。

当向该结构中:

- 插入元素

根据待插入元素的关键码, 以此函数计算出该元素的存储位置并按此位置进行存放

- 搜索元素

对元素的关键码进行同样的计算, 把求得的函数值当做元素的存储位置, 在结构中按此位置取元素比较, 若关键码相等, 则搜索成功

该方式即为哈希(散列)方法, 哈希方法中使用的转换函数称为哈希(散列)函数, 构造出来的结构称为哈希表(Hash Table)(或者称散列表)

例如: 数据集合{1, 7, 6, 4, 5, 9};

哈希函数设置为: $\text{hash}(\text{key}) = \text{key} \% \text{capacity}$; capacity为存储元素底层空间总的大小。

哈希函数: $\text{hash}(\text{key}) = \text{key} \% \text{capacity}$ $\text{capacity} = 10$

0	1	2	3	4	5	6	7	8	9
	1			4	5	6	7		9

$\text{hash}(1) = 1 \% 10 = 1$ $\text{hash}(7) = 7 \% 10 = 7$ $\text{hash}(6) = 6 \% 10 = 6$

$\text{hash}(4) = 4 \% 10 = 4$ $\text{hash}(5) = 5 \% 10 = 5$ $\text{hash}(9) = 9 \% 10 = 9$

用该方法进行搜索不必进行多次关键码的比较, 因此搜索的速度比较快 问题: 按照上述哈希方式, 向集合中插入元素44, 会出现什么问题?

5.2 冲突-概念

对于两个数据元素的关键字 k_i 和 k_j ($i \neq j$), 有 $k_i \neq k_j$, 但有: $\text{Hash}(k_i) = \text{Hash}(k_j)$, 即: 不同关键字通过相同哈希数计算出相同的哈希地址, 该种现象称为哈希冲突或哈希碰撞。

把具有不同关键码而具有相同哈希地址的数据元素称为“同义词”。

5.3 冲突-避免

首先, 我们需要明确一点, 由于我们哈希表底层数组的容量往往是小于实际要存储的关键字的数量的, 这就导致一个问题, 冲突的发生是必然的, 但我们能做的应该是尽量降低冲突率。

5.4 冲突-避免-哈希函数设计

引起哈希冲突的一个原因可能是: 哈希函数设计不够合理。 哈希函数设计原则:

- 哈希函数的定义域必须包括需要存储的全部关键码, 而如果散列表允许有 m 个地址时, 其值域必须在0到 $m-1$ 之间
- 哈希函数计算出来的地址能均匀分布在空间中
- 哈希函数应该比较简单

常见哈希函数

1. 直接定址法--(常用)

取关键字的某个线性函数为散列地址: $\text{Hash}(\text{Key}) = A * \text{Key} + B$ 优点: 简单、均匀 缺点: 需要事先知道关键字的分布情况 使用场景: 适合查找比较小且连续的情况 面试题: [字符串中第一个只出现一次字符](#)

2. 除留余数法--(常用)

设散列表中允许的地址数为 m , 取一个不大于 m , 但最接近或者等于 m 的质数 p 作为除数, 按照哈希函数: $\text{Hash}(\text{key}) = \text{key} \% p$ ($p \leq m$), 将关键码转换成哈希地址

3. 平方取中法--(了解)

假设关键字为1234, 对它平方就是1522756, 抽取中间的3位227作为哈希地址; 再比如关键字为4321, 对它平方就是18671041, 抽取中间的3位671(或710)作为哈希地址 平方取中法比较适合: 不知道关键字的分布, 而位数又不是很大的情况

4. 折叠法--(了解)

折叠法是将关键字从左到右分割成位数相等的几部分(最后一部分位数可以短些),然后将这几部分叠加求和,并按散列表表长,取后几位作为散列地址。

折叠法适合事先不需要知道关键字的分布, 适合关键字位数比较多的情况

5. 随机数法--(了解)

选择一个随机函数, 取关键字的随机函数值为它的哈希地址, 即 $H(\text{key}) = \text{random}(\text{key})$, 其中 random 为随机数函数。

通常应用于关键字长度不等时采用此法

6. 数学分析法--(了解)

设有 n 个 d 位数, 每一位可能有 r 种不同的符号, 这 r 种不同的符号在各位上出现的频率不一定相同, 可能在某些位上分布比较均匀, 每种符号出现的机会均等, 在某些位上分布不均匀只有某几种符号经常出现。可根据散列表的大小, 选择其中各种符号分布均匀的若干位作为散列地址。例如:

130xxxx1234
130xxxx2345
138xxxx4829
138xxxx2396
138xxxx8354

易重复分布太集中某几个数字 分布均匀, 可用作散列地址

假设要存储某家公司员工登记表, 如果用手机号作为关键字, 那么极有可能前7位都是相同的, 那么我们可以选择后面的四位作为散列地址, 如果这样的抽取工作还容易出现冲突, 还可以对抽取出来的数字进行反转(如1234改成4321)、右环位移(如1234改成4123)、左环移位、前两数与后两数叠加(如1234改成 $12+34=46$)等方法。

数字分析法通常适合处理关键字位数比较大的情况, 如果事先知道关键字的分布且关键字的若干位分布较均匀的情况

注意: 哈希函数设计的越精妙, 产生哈希冲突的可能性就越低, 但是无法避免哈希冲突

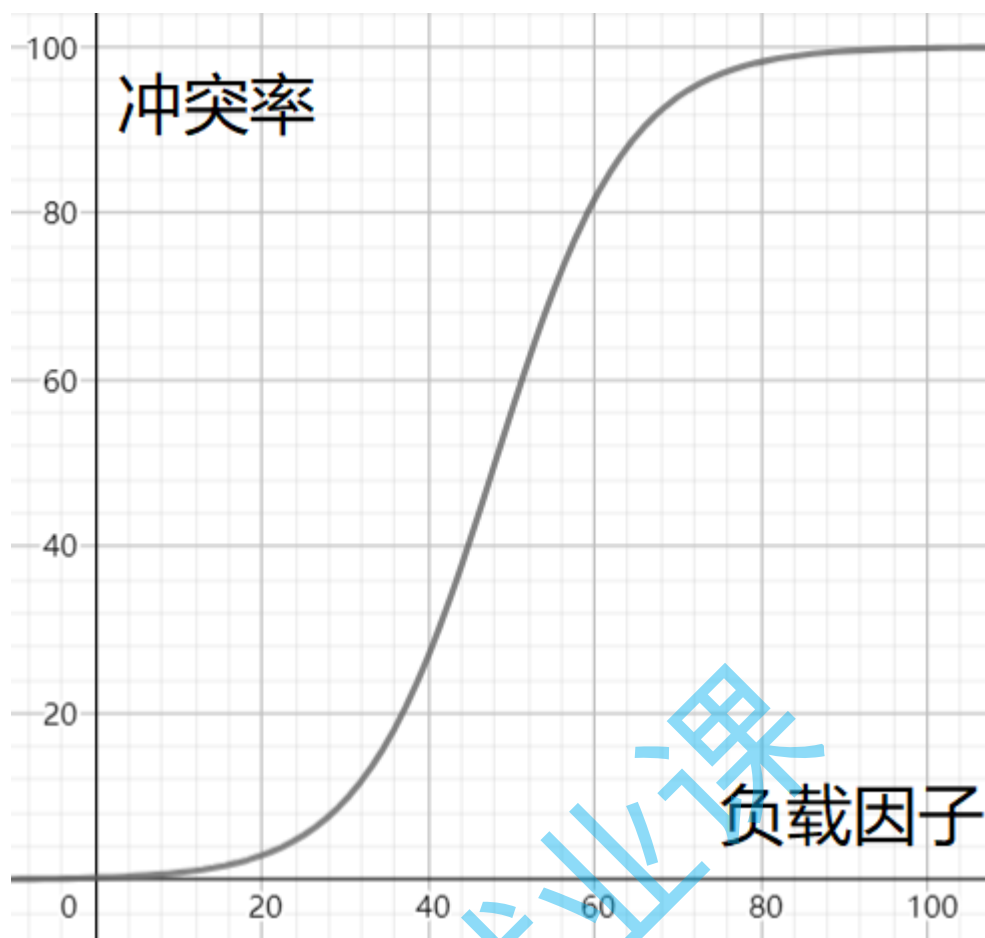
5.5 冲突-避免-负载因子调节 (重点掌握)

散列表的载荷因子定义为: $\alpha = \text{填入表中的元素个数} / \text{散列表的长度}$

α 是散列表装满程度的标志因子。由于表长是定值, α 与“填入表中的元素个数”成正比, 所以, α 越大, 表明填入表中的元素越多, 产生冲突的可能性就越大; 反之, α 越小, 表明填入表中的元素越少, 产生冲突的可能性就越小。实际上, 散列表的平均查找长度是载荷因子 α 的函数, 只是不同处理冲突的方法有不同的函数。

对于开放定址法, 载荷因子是特别重要因素, 应严格限制在0.7-0.8以下。超过0.8, 查表时的CPU缓存不命中(cache missing)按照指数曲线上升。因此, 一些采用开放定址法的hash库, 如Java的系统库限制了载荷因子为0.75, 超过此值将resize散列表。

负载因子和冲突率的关系粗略演示



所以当冲突率达到了一个无法忍受的程度时，我们需要通过降低负载因子来变相的降低冲突率。

已知哈希表中已有的关键字个数是不可变的，那我们能调整的就只有哈希表中的数组的大小。

5.6 冲突-解决

解决哈希冲突两种常见的方法是：闭散列和开散列

5.7 冲突-解决-闭散列

闭散列：也叫开放定址法，当发生哈希冲突时，如果哈希表未被装满，说明在哈希表中必然还有空位置，那么可以把key存放到冲突位置中的“下一个”空位置中去。那如何寻找下一个空位置呢？

1. 线性探测

比如上面的场景，现在需要插入元素44，先通过哈希函数计算哈希地址，下标为4，因此44理论上应该插在该位置，但是该位置已经放了值为4的元素，即发生哈希冲突。

线性探测：从发生冲突的位置开始，依次向后探测，直到寻找到下一个空位置为止。

○ 插入

- 通过哈希函数获取待插入元素在哈希表中的位置
- 如果该位置中没有元素则直接插入新元素，如果该位置中有元素发生哈希冲突，使用线性探测找到下一个空位置，插入新元素

哈希函数: $\text{hash}(\text{key}) = \text{key} \% \text{capacity}$ $\text{capacity} = 10$

0	1	2	3	4	5	6	7	8	9
	1			4	5	6	7	44	9

$\text{hash}(1) = 1 \% 10 = 1$ $\text{hash}(7) = 7 \% 10 = 7$ $\text{hash}(6) = 6 \% 10 = 6$

$\text{hash}(4) = 4 \% 10 = 4$ $\text{hash}(5) = 5 \% 10 = 5$ $\text{hash}(9) = 9 \% 10 = 9$

- 采用闭散列处理哈希冲突时, 不能随便物理删除哈希表中已有的元素, 若直接删除元素会影响其他元素的搜索。比如删除元素4, 如果直接删除掉, 44查找起来可能会受影响。因此线性探测采用标记的伪删除法来删除一个元素。

2. 二次探测

线性探测的缺陷是产生冲突的数据堆积在一块, 这与其找下一个空位置有关系, 因为找空位置的方式就是挨着往后逐个去找, 因此二次探测为了避免该问题, 找下一个空位置的方法为: $H_i = (H_0 + i^2) \% m$, 或者: $H_i = (H_0 - i^2) \% m$ 。其中: $i = 1, 2, 3, \dots$, H_0 是通过散列函数 $\text{Hash}(x)$ 对元素的关键码 key 进行计算得到的位置, m 是表的大小。对于2.1中如果要插入44, 产生冲突, 使用解决后的情况为:

哈希函数: $\text{hash}(\text{key}) = \text{key} \% \text{capacity}$ $\text{capacity} = 10$

0	1	2	3	4	5	6	7	8	9
	1			4	5	6	7	44	9

$\text{hash}(1) = 1 \% 10 = 1$ $\text{hash}(7) = 7 \% 10 = 7$ $\text{hash}(6) = 6 \% 10 = 6$

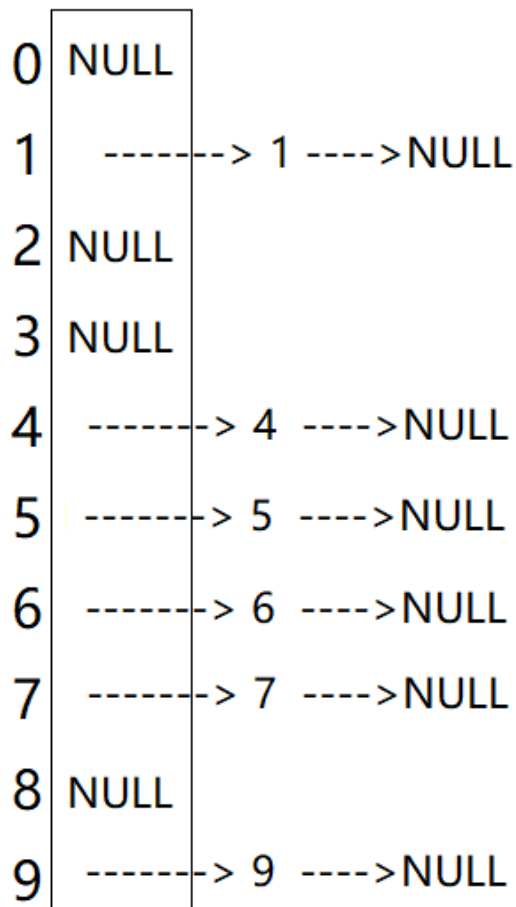
$\text{hash}(4) = 4 \% 10 = 4$ $\text{hash}(5) = 5 \% 10 = 5$ $\text{hash}(9) = 9 \% 10 = 9$

研究表明: 当表的长度为质数且表装载因子 a 不超过0.5时, 新的表项一定能够插入, 而且任何一个位置都不会被探查两次。因此只要表中有一半的空位置, 就不会存在表满的问题。在搜索时可以不考虑表装满的情况, 但在插入时必须确保表的装载因子 a 不超过0.5, 如果超出必须考虑增容。

因此: 比散列最大的缺陷就是空间利用率比较低, 这也是哈希的缺陷。

5.8 冲突-解决-开散列/哈希桶 (重点掌握)

开散列法又叫链地址法(开链法), 首先对关键码集合用散列函数计算散列地址, 具有相同地址的关键码归于同一子集合, 每一个子集合称为一个桶, 各个桶中的元素通过一个单链表链接起来, 各链表的头结点存储在哈希表中。



$\text{hash}(\text{key}) = \text{key} \% \text{capacity}$
 $\text{capacity} = 10$

从上图可以看出，开散列中每个桶中放的都是发生哈希冲突的元素。

开散列，可以认为是把一个在大集合中的搜索问题转化为在小集合中做搜索了。

5.9 冲突严重时的解决办法

刚才我们提到了，哈希桶其实可以看作将大集合的搜索问题转化为小集合的搜索问题了，那如果冲突严重，就意味着小集合的搜索性能其实也时不佳的，这个时候我们就可以将这个所谓的小集合搜索问题继续进行转化，例如：

1. 每个桶的背后是另一个哈希表
2. 每个桶的背后是一棵搜索树

5.10

5.10 实现

```
// key-value 模型
public class HashBucket {
    private static class Node {
        private int key;
        private int value;
        Node next;

        public Node(int key, int value) {
            this.key = key;
            this.value = value;
        }
    }
}
```

```

}

private Node[] array;
private int size; // 当前的数据个数
private static final double LOAD_FACTOR = 0.75;

public int put(int key, int value) {
    int index = key % array.length;

    // 在链表中查找 key 所在的结点
    // 如果找到了, 更新
    // 所有结点都不是 key, 插入一个新的结点
    for (Node cur = array[index]; cur != null; cur = cur.next) {
        if (key == cur.key) {
            int oldValue = cur.value;
            cur.value = value;
            return oldValue;
        }
    }
    Node node = new Node(key, value);
    node.next = array[index];
    array[index] = node;
    size++;

    if (loadFactor() >= LOAD_FACTOR) {
        resize();
    }

    return -1;
}

private void resize() {
    Node[] newArray = new Node[array.length * 2];
    for (int i = 0; i < array.length; i++) {
        Node next;
        for (Node cur = array[i]; cur != null; cur = next) {
            next = cur.next;
            int index = cur.key % newArray.length;
            cur.next = newArray[index];
            newArray[index] = cur;
        }
    }
    array = newArray;
}

private double loadFactor() {
    return size * 1.0 / array.length;
}

public HashBucket() {
    array = new Node[8];
    size = 0;
}

```

```
public int get(int key) {
    int index = key % array.length;

    Node head = array[index];
    for (Node cur = head; cur != null; cur = cur.next) {
        if (key == cur.key) {
            return cur.value;
        }
    }

    return -1;
}
```

5.11 性能分析

虽然哈希表一直在和冲突做斗争，但在实际使用过程中，我们认为哈希表的冲突率是不高的，冲突个数是可控的，也就是每个桶中的链表的长度是一个常数，所以，通常意义下，我们认为**哈希表的插入/删除/查找时间复杂度是 $O(1)$** 。

5.12 和 java 类集的关系

1. HashMap 和 HashSet 即 java 中利用哈希表实现的 Map 和 Set
2. java 中使用的是哈希桶方式解决冲突的
3. java 会在冲突链表长度大于一定阈值后，将链表转变为搜索树（红黑树）
4. java 中计算哈希值实际上是调用的类的 hashCode 方法，进行 key 的相等性比较是调用 key 的 equals 方法。所以如果要用自定义类作为 HashMap 的 key 或者 HashSet 的值，**必须覆写 hashCode 和 equals 方法**，而且要做到 equals 相等的对象，hashCode 一定是一致的。

6 OJ练习

1. [只出现一次的数字](#)
2. [复制带随机指针的链表](#)
3. [宝石与石头](#)
4. [坏键盘打字](#)
5. [前K个高频单词](#)